

UNIVERSITÀ DI BOLOGNA



School of Engineering
Master Degree in Automation Engineering

Distributed Autonomous Systems
Group 19

Professors:
Giuseppe Notarstefano
Ivano Notarnicola

Students:
Romeo Simone

Academic year 2024/2025

Abstract

This report documents the implementation of two algorithms for multi-robot systems. The first task tackle the cooperative localization of some targets using consensual optimization, showing how graph connectivity and number of robots influence convergence and accuracy. The second task implements an aggregative optimization algorithm that balances private goals and fleet compactness, both in Python and in ROS 2. The results highlight the efficiency of this technique by testing it in several simulations.

Contents

Introduction	6
1 Task 1 - Multi-Robot Target Localization	7
1.1 Task 1.1 - Distributed Consensus Optimization	7
1.1.1 Problem Description	7
1.1.2 Gradient Tracking Algorithm	7
1.1.3 Assumptions for Convergence	8
1.1.4 Parameters Selection	9
1.1.5 Simulations	9
1.1.6 Insight on Number of Agents	12
1.1.7 Conclusion	14
1.2 Task 1.2 - Cooperative multi-robot target localization	14
1.2.1 Problem Formulation	14
1.2.2 Parameters Selection	15
1.2.3 Simulations	15
1.2.4 Conclusion	18
2 Task 2 - Aggregative Optimization for Multi-Robot Systems	19
2.1 Task 2.1 - Distributed aggregative optimization	19
2.1.1 Aggregative Tracking Algorithm	19
2.1.2 Assumptions for Convergence	20
2.1.3 Parameters Selection	20
2.1.4 Simulations	21
2.1.5 Insight on β	23
2.1.6 Conclusion	26
2.2 Task 2.2 - ROS 2 Implementation	26
2.2.1 Workflow	26
2.2.2 Robot Nodes	26
2.2.3 Visualizer Node and RViz	26
2.2.4 Communication Protocol	26
2.2.5 Simulations	27
2.2.6 Comparison with Task 2.1	29
2.2.7 Conclusion	30

Conclusions	31
Bibliography	32

Introduction

This report describes the implementation of two distributed optimization tasks for multi-robot systems with N robots. The project is split in two parts, each one tackling a different scenario and algorithm:

Task 1. Multi-robot target localization using distributed consensus optimization.

Task 2. Aggregative optimization for coordinated robot movement.
The following chapters provide analysis and results for each task.

Chapter 1

Task 1 - Multi-Robot Target Localization

The scenario is the following: there is a fleet of $N \in \mathbb{N}$ robots that has to cooperatively localize randomly defined $N_T \in \mathbb{N}$ targets. Each robot performs a noisy measure of the targets' location. The goal is to implement the distributed gradient tracking algorithm and then to use it to correctly localize the targets.

1.1 Task 1.1 - Distributed Consensus Optimization

1.1.1 Problem Description

This task addresses the implementation and evaluation of the Gradient Tracking algorithm to solve a consensus optimization problem. Each agent i has a local quadratic cost function:

$$\ell_i(z) = \frac{1}{2}z^\top Qz + r^\top z$$

where $Q \in \mathbb{R}^{d \times d}$ is symmetric positive definite and $r \in \mathbb{R}^d$.

The goal is to solve:

$$\min_{z \in \mathbb{R}^d} \sum_{i=1}^N \ell_i(z)$$

with N the number of agents and $d > 1$.

1.1.2 Gradient Tracking Algorithm

The gradient tracking algorithm has been implemented as follows. Each agent maintains an estimate z_i of the decision variable and a local

tracker s_i of the gradient. The update rules are:

$$\begin{aligned} z_i^{k+1} &= \sum_{j \in \mathcal{N}_i} w_{ij} z_j^k - \alpha s_i^k \\ s_i^{k+1} &= \sum_{j \in \mathcal{N}_i} w_{ij} s_j^k + \nabla \ell_i(z_i^{k+1}) - \nabla \ell_i(z_i^k) \end{aligned}$$

where W is a weight matrix computed with Metropolis-Hastings rule, and α is a constant step-size.

1.1.3 Assumptions for Convergence

Assumption 1: Let $w_{ij}, i, j \in \{1, \dots, N\}$ be nonnegative entries of the weight matrix W associated to the undirected and connected graph G , with $w_{ii} > 0$ and W doubly stochastic.

Assumption 2: For all $i \in \{1, \dots, N\}$, each cost function $\ell_i : \mathbb{R}^d \rightarrow \mathbb{R}$ satisfies the following conditions:

ℓ_i is strongly convex with coefficient $\mu > 0$

ℓ_i has Lipschitz continuous gradient with constant $L > 0$

Theorem: If these assumptions hold, there exists $\alpha^* > 0$ such that for all $\alpha \in (0, \alpha^*)$ the sequences of local solution estimates $\{z_i^k\}_{k \in \mathbb{N}}, i = 1, \dots, N$, generated by the Gradient Tracking algorithm, asymptotically converge to a (consensual) solution z^* of the problem, i.e., for all $i = 1, \dots, N$, it holds:

$$\lim_{k \rightarrow \infty} \|z_i^k - z^*\| = 0 \quad (1.1)$$

Moreover, the convergence rate is linear and the stability is exponential, that means that for all $i = 1, \dots, N$ exists $\rho \in (0, 1)$ such that:

$$\|z_i^k - z^*\| \leq \rho \|z_i^{k-1} - z^*\| \leq \rho^k \|z_i^0 - z^*\| \quad (1.2)$$

for all $k \in \mathbb{N}$. [1]

1.1.4 Parameters Selection

The following parameters are kept constant for the next simulations, unless differently specified:

Number of agents	$N = 10$
Dimension	$d = 5$
Local cost	random Q_i (positive definite) and r_i sampled from a standard normal distribution
Step-size	0.01
Maximum iterations	10000
Graph topologies	cycle, path, star, ER

1.1.5 Simulations

In this section, some simulations are performed varying the graph topology.

Total Cost Evolution

This metric describes the evolution of the cost J over the iterations:

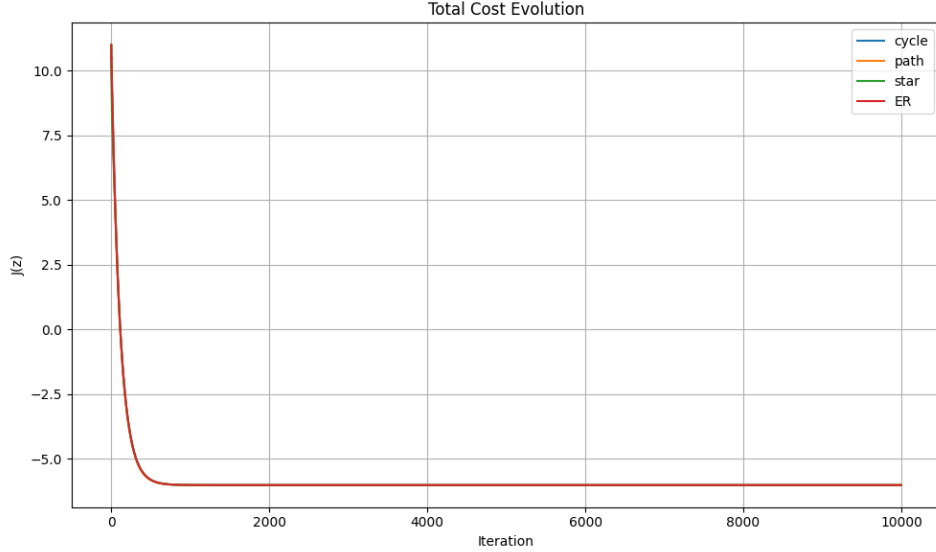


Figure 1.1: Total Cost Evolution

We can observe that the plots coincide perfectly. Changing topology does not lead to any variation.

In every simulation, the cost quickly converges to a negative value. This is

plausible as, although the matrix Q is positive definite, the vector r can be negative. If negative enough, this can lead the final cost to be negative.

Total Cost Error

This metric describes the error $|J - J^*|$ over the iterations:

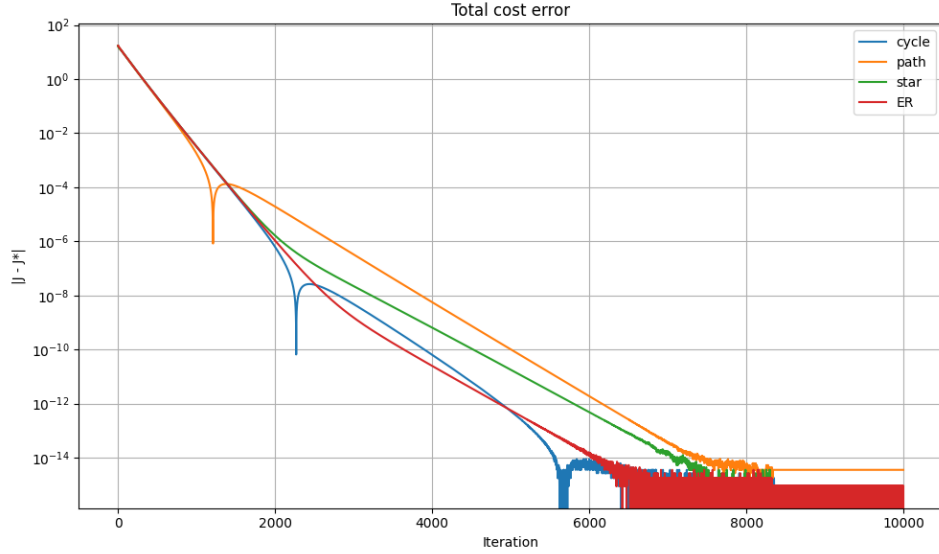


Figure 1.2: Total Cost Error

Much more interesting is the total cost error plot. In each case, the error basically converges to zero. In some plots, we can notice that there are some spikes, but in a logarithmic scale this may happen if there is a quick variation in the error value.

The graph topologies that perform better (in terms of convergence time) are cycle and ER, that converge to zero in about 6500 iterations. This is probably due to the graph being more connected, that assures a faster convergence rate. On the same page, the graph topologies that perform worse are star and path, that are not greatly connected topologies. These two converges in more or less 8000 iterations.

Total Gradient Norm

This metric describes the norm of the gradient $|\sum_i \nabla \ell_i(z_i)|$ over the iterations:

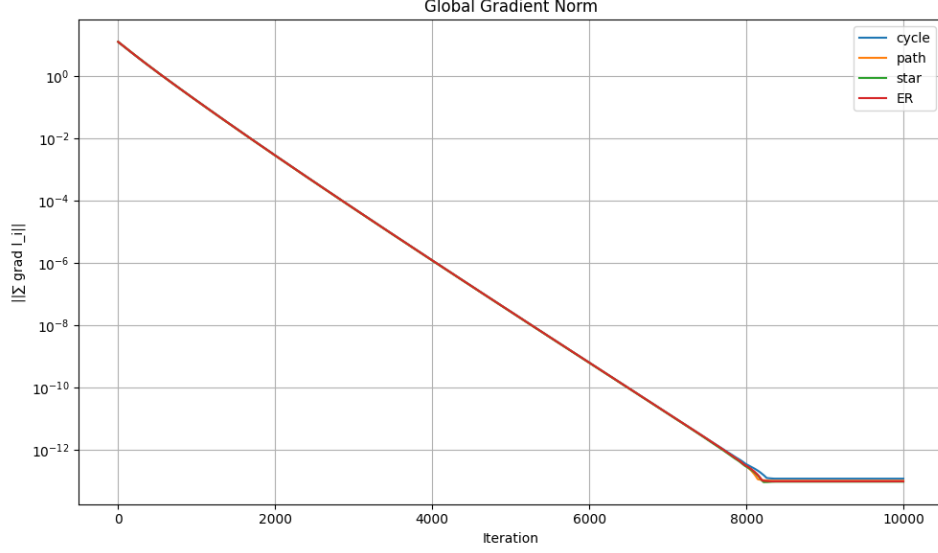
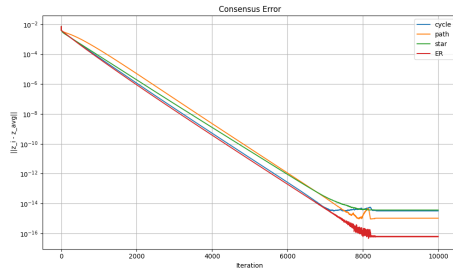


Figure 1.3: Total Gradient Norm

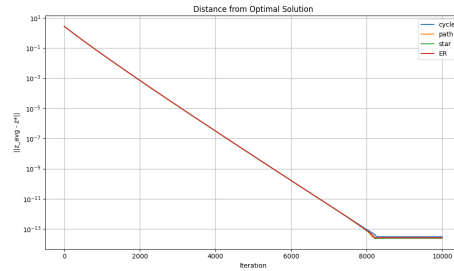
For the total gradient norm, each plot converges to zero in slightly more than 8000 iterations. Again, changing topology does not lead to any difference.

Consensus Error and Distance from Optimal Solution

These metrics describe the consensus error and the distance from the optimal solution over the iterations:



(a) Consensus Error



(b) Distance from Optimal Solution

The consensus error shows how each agent agrees with the other agents, relating their prediction with the average of all agents. In this case, it

converges to zero with similar rates across all topologies. This shows that the agents agree on the final value of the prediction.

The distance from optimal solution is complementary to the previous plot. If the consensus error shows if the agents agree, this error shows if the agreed value is correct. In this task we could solve the optimization problem and obtain J^* , that allowed to plot this graph. We can conclude that, not only the agents agree on the value, but that value is also close to the optimal solution for the problem.

1.1.6 Insight on Number of Agents

Until now, we kept a fixed number of agents (10). It may be interesting to understand what are the differences with less or more agents. We will repeat the simulations with 3 and 20 agents. For sake of simplicity, we will use only a topology, this being cycle.

Total Cost Evolution

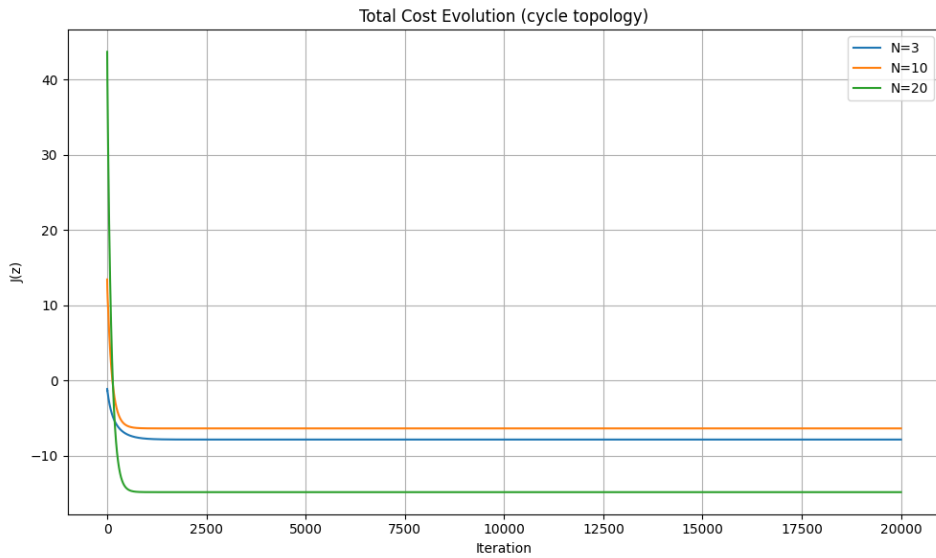


Figure 1.5: Total Cost Evolution

Total Cost Error



Figure 1.6: Total Cost Error

Global Gradient Norm

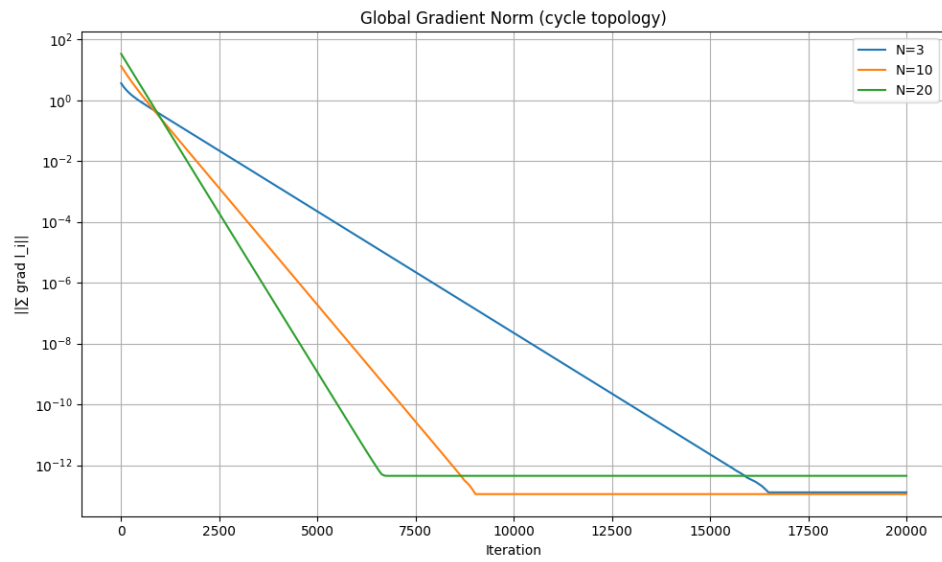
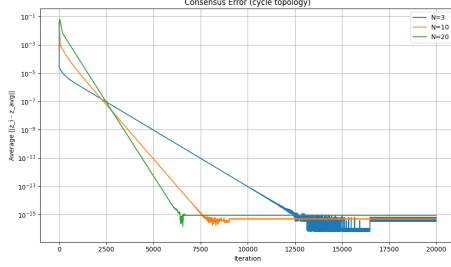
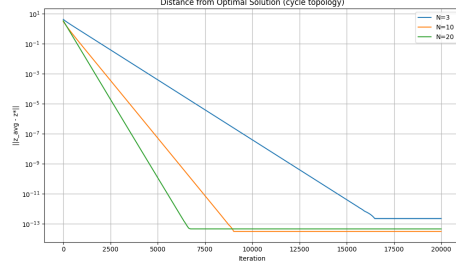


Figure 1.7: Global Gradient Norm

Consensus Error and Distance from Optimal Solution



(a) Consensus Error



(b) Distance from Optimal Solution

From the additional insight on the changes upon varying the number of agents, we can conclude that more agents lead to faster convergence. In fact, every metrics converge faster in the case of $N = 20$, while slower if $N = 3$.

1.1.7 Conclusion

The gradient tracking algorithm was successfully implemented and we can conclude that the more the graph is connected, the better the outcome. Also, more agents lead to faster convergence.

1.2 Task 1.2 - Cooperative multi-robot target localization

1.2.1 Problem Formulation

Each robot i at position p_i measures noisy distance to targets τ :

$$d_{i\tau} = \|p_i - z_\tau\| + v, \quad v \sim \mathcal{N}(0, \sigma^2) \quad (1.3)$$

Local cost function for target estimation:

$$\ell_i(z) = \sum_{\tau=1}^{N_T} (d_{i\tau}^2 - \|z_\tau - p_i\|^2)^2 \quad (1.4)$$

1.2.2 Parameters Selection

The following parameters are kept constant for the next simulations, unless differently specified:

Number of agents	$N = 10$
Number of targets	$N_T = 3$
Measurement noise	$\sigma = 0.1$
Step-size	0.01
Maximum iterations	10000
Graph topologies	cycle, path, star, ER

1.2.3 Simulations

In this section, some simulations are performed varying the graph topology.

Total Cost Evolution

This metric describes the evolution of the cost J over the iterations:

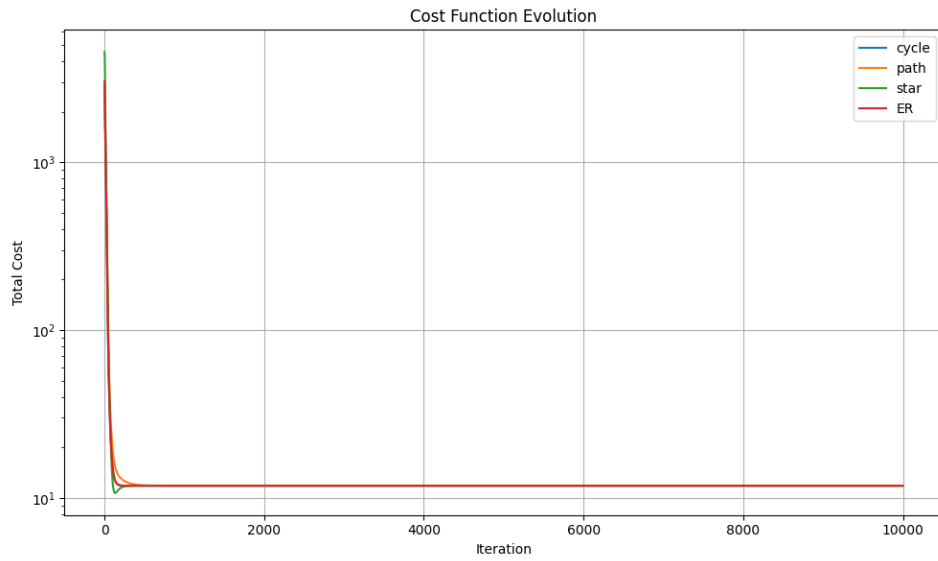


Figure 1.9: Total Cost Evolution

There are no significant differences in the total cost evolution between each topology.

Total Gradient Norm

This metric describes the norm of the gradient $|\sum_i \nabla \ell_i(z_i)|$ over the iterations:

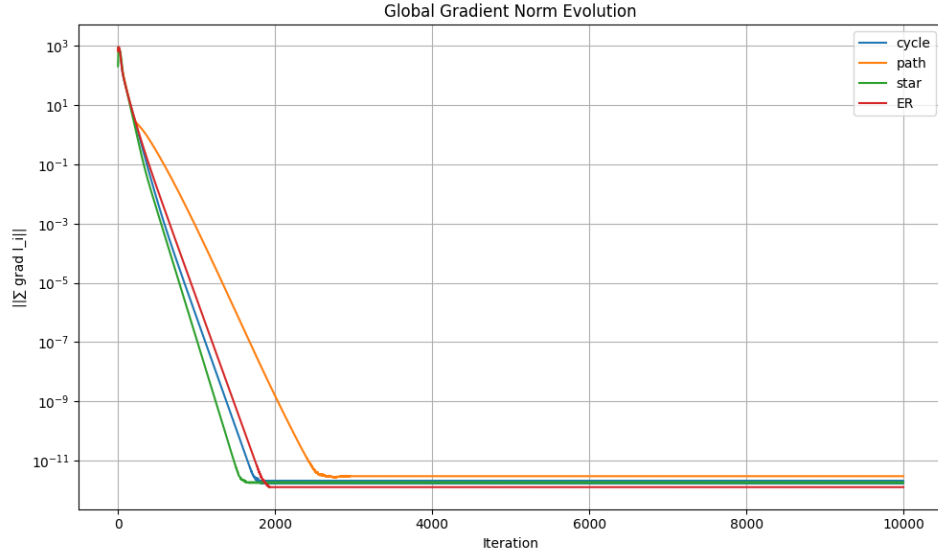


Figure 1.10: Total Gradient Norm

Instead, for the gradient norm evolution, it is possible to notice that star topology is the first to converge (≈ 1500 iterations), while the last one is path (≈ 2500 iterations).

Target Position Estimation Error

This metric describes the average Euclidean error between estimated and true target positions, computed across all robots and targets vs. iterations:

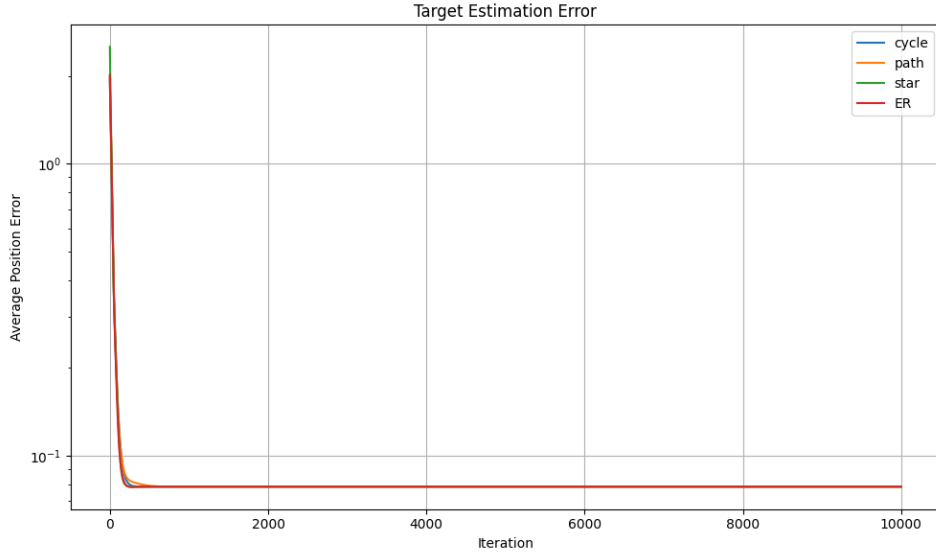
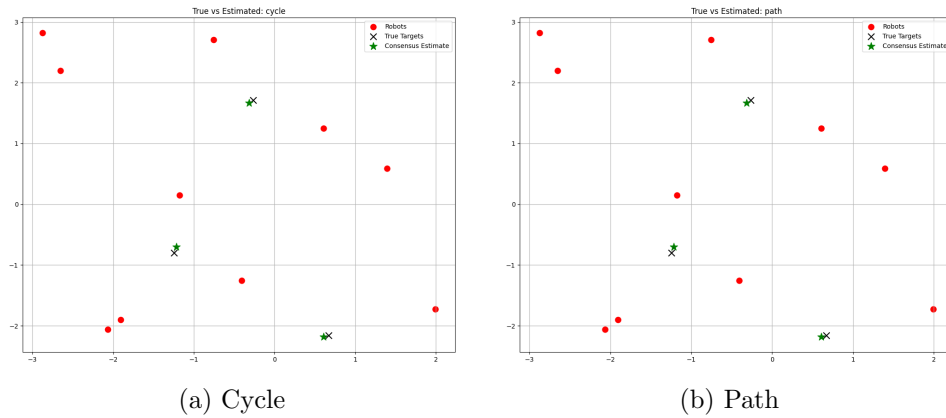


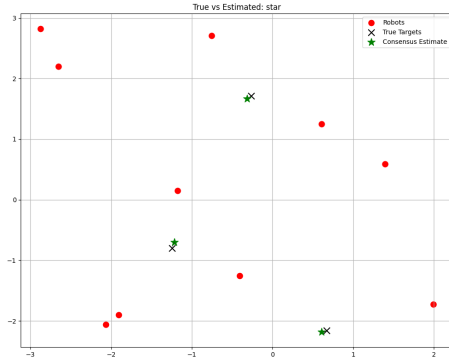
Figure 1.11: Total Cost Error

Like cost evolution, also the target position estimation error is basically the same for each graph topology.

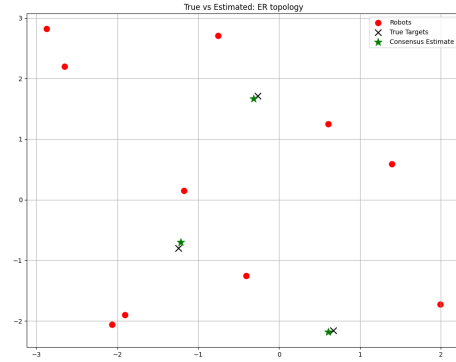
Final True vs Estimated Target Positions

This is a scatter plot that shows the final estimate for target positions and the real target positions:





(a) Star



(b) ER

Even after a close analysis, it looks like the estimations are very similar for each topology. To verify this last result, here follows the final values of target position estimation error for each topology:

- Cycle: 0.0785
- Path: 0.0785
- Star: 0.0785
- ER: 0.0785
- Cycle (3 agents): 0.131
- Cycle (20 agents): 0.0355
- Cycle (50 agents): 0.0254

Even without being approximated, the error is almost identical if the number of agents is not changed. Instead, when the number of agents increases the error decreases and vice-versa.

1.2.4 Conclusion

If there are enough agents, we are able to reconstruct the position of the targets with a low error. Also with 10 agents, we are still getting a good reconstruction given that we set $\sigma = 0.1$.

Chapter 2

Task 2 - Aggregative Optimization for Multi-Robot Systems

In this task, we implement a distributed control strategy for a team of N robots that must balance two objectives:

1. Stay close to private targets $t_i \in \mathbb{R}^2$
2. Stay close to the barycenter $\sigma(z) = \frac{1}{N} \sum_{i=1}^N z_i$

The optimization problem is formulated as:

$$\min_{z_1, \dots, z_N} \sum_{i=1}^N \ell_i(z_i, \sigma(z)) \quad (2.1)$$

where the local cost function for each robot is:

$$\ell_i(z_i, \sigma(z)) = \frac{1}{2} \|z_i - t_i\|^2 + \frac{\beta}{2} \|z_i - \sigma(z)\|^2 \quad (2.2)$$

with $\beta > 0$ being a trade-off parameter between the two objectives. For Task 2.1 we will see an implementation of this algorithm, while Task 2.2 will be the ROS implementation of the same code.

2.1 Task 2.1 - Distributed aggregative optimization

2.1.1 Aggregative Tracking Algorithm

As we are in a distributed setting, we cannot know global measures, but only local ones. For this reason, aggregative tracking algorithm uses local

estimates s_i^k for the barycenter and v_i^k for the gradient.
In particular, the updates are the following:

$$z_i^{k+1} = z_i^k - \alpha \left[(z_i^k - t_i) + \beta(z_i^k - s_i^k) + v_i^k \right] \quad (2.3)$$

$$s_i^{k+1} = \sum_{j \in \mathcal{N}_i} w_{ij} s_j^k + (z_i^{k+1} - z_i^k) \quad (2.4)$$

$$v_i^{k+1} = \sum_{j \in \mathcal{N}_i} w_{ij} v_j^k + \nabla_2 \ell_i^{k+1} - \nabla_2 \ell_i^k \quad (2.5)$$

where, the gradients are:

$$\nabla_1 \ell_i^k = (z_i^k - t_i) + \beta(z_i^k - s_i^k) \quad (2.6)$$

$$\nabla_2 \ell_i^k = v_i^k \quad (2.7)$$

2.1.2 Assumptions for Convergence

Assumption on the graph: Let G be a strongly connected and aperiodic digraph

Assumption on the weight matrix: Let W be doubly stochastic.

Assumption on the functions: The following conditions are satisfied:

1. $\sum_{i=1}^N \ell_i(\cdot, \sigma(\cdot))$ is strongly convex
2. ϕ_i is differentiable and Lipschitz continuous
3. $\nabla_1 \ell_i(\cdot, \cdot)$, $\nabla_2 \ell_i(\cdot, \cdot)$ and $\nabla \phi_i(\cdot) \nabla_2 \ell_i(\cdot, \cdot)$ are Lipschitz continuous

Theorem: If these assumptions hold, there exists $\alpha^* > 0$ such that for all $\alpha \in (0, \alpha^*)$ the sequences of local solution estimates $\{z_1^k, \dots, z_N^k\}_{k \in \mathbb{N}}$, generated by the Aggregative Tracking distributed optimization algorithm, satisfy:

$$\lim_{k \rightarrow \infty} \|z_i^k - z_i^*\| = 0 \quad (2.8)$$

at a linear rate, for all $i = 1, \dots, N$. [1]

2.1.3 Parameters Selection

The following parameters are kept constant for the next simulations, unless differently specified:

Number of agents	$N = 5$
Step-size	$\alpha = 0.01$
Tradeoff parameter	$\beta = 0.5$
Maximum iterations	5000
Graph topologies	cycle, path, star, ER

2.1.4 Simulations

In this section, some simulations are performed varying the graph topology.

Total Cost Evolution

This metric describes the evolution of the cost J over the iterations:

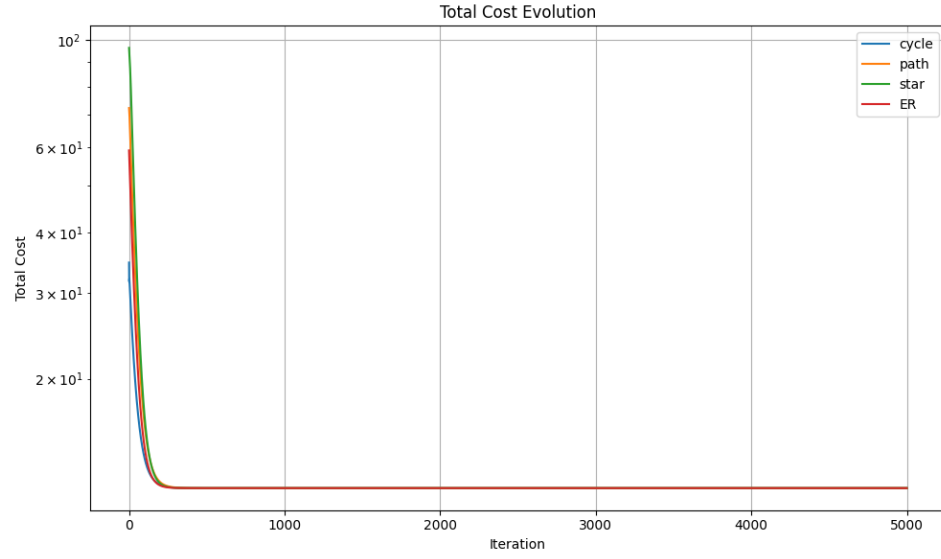


Figure 2.1: Total Cost Evolution

There are no significant differences in the total cost evolution between each topology.

Global Gradient Norm

This metric describes the norm of the gradient $|\sum_i \nabla \ell_i(z_i)|$ over the iterations:

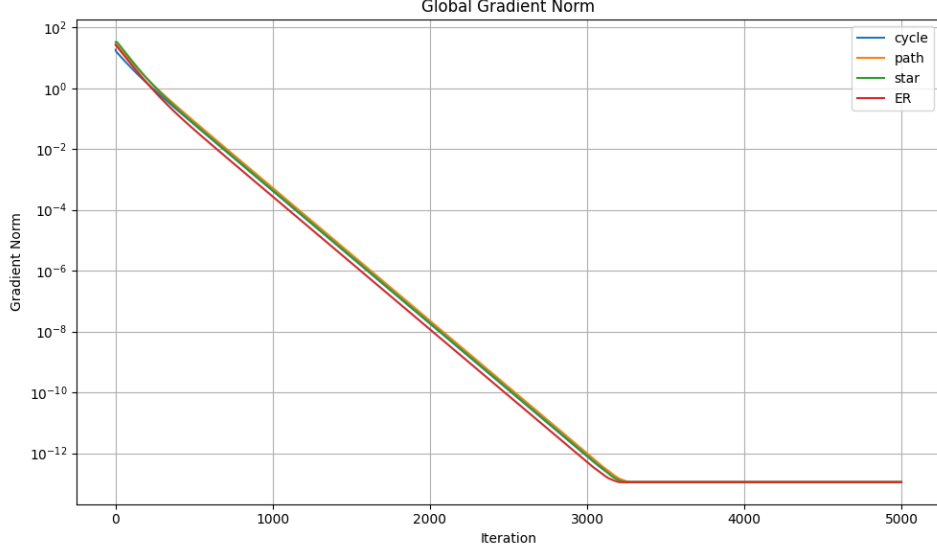
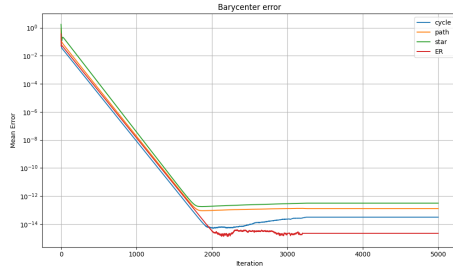


Figure 2.2: Total Gradient Norm

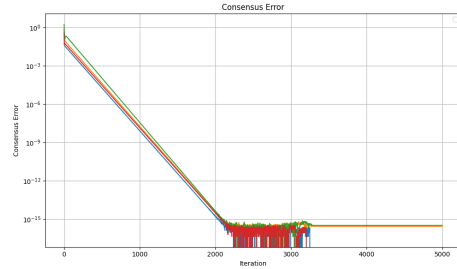
For the gradient norm evolution, it is possible to notice that each topology converge at approximately 3200 iterations.

Barycenter and Consensus Error

These metrics describe the barycenter and the consensus error over the iterations. The barycenter error is the average of difference between barycenter estimates and true barycenter. The consensus error is the difference between barycenter estimate for each agent and the average barycenter.



(a) Barycenter Error



(b) Consensus Error

Barycenter and consensus error are complementary. While the first one shows that the estimated barycenter converges to the real one, the second

shows that the robots agree on the estimated barycenter position. For both the metrics, the results are comparable between each topology. Probably due to lack of connections, sometimes the ER topology does not have a large consensus. Examining further, I noticed that sometimes the estimates are indeed different.

The true barycenter is:

$$\begin{bmatrix} -1.15650 & 1.55923 \end{bmatrix}$$

The barycenter estimates for each agent, with topologies cycle, path and star coincide:

$$\begin{bmatrix} -1.15650 & 1.55923 \\ -1.15650 & 1.55923 \\ -1.15650 & 1.55923 \\ -1.15650 & 1.55923 \\ -1.15650 & 1.55923 \end{bmatrix}$$

Finally, the barycenter estimates for the ER topology are:

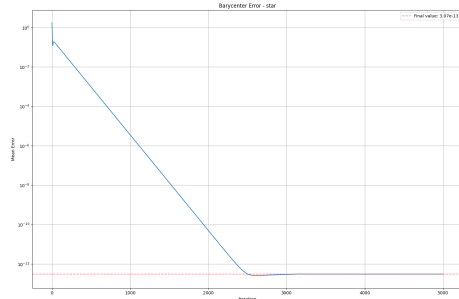
$$\begin{bmatrix} -1.69841 & 1.42886 \\ -1.69841 & 1.42886 \\ -1.69841 & 1.42886 \\ -1.69841 & 1.42886 \\ 1.01115 & 2.08073 \end{bmatrix}$$

Even though the estimates are different in ER, the mean of the rows gives exactly the true barycenter vector, that is the reason why the barycenter does converge.

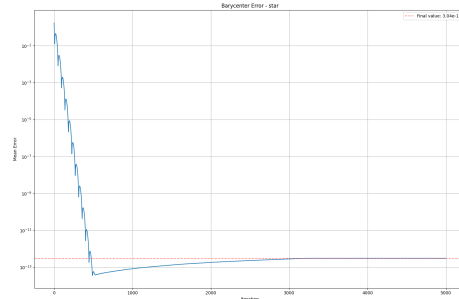
2.1.5 Insight on β

Let us analyze only the star topology for this section. Changing β leads to the following outcome:

Barycenter Error

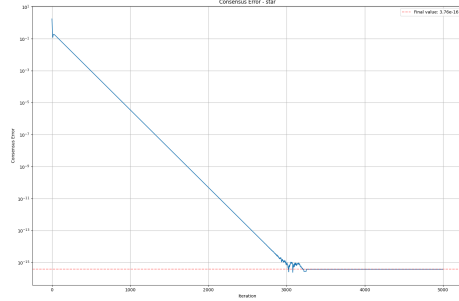
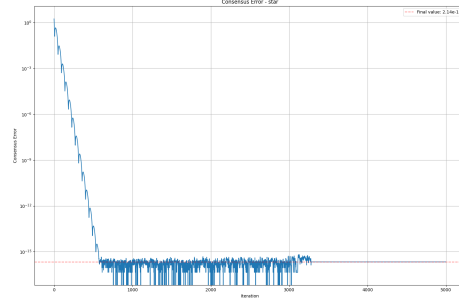


(a) $\beta = 0.1$



(b) $\beta = 5$

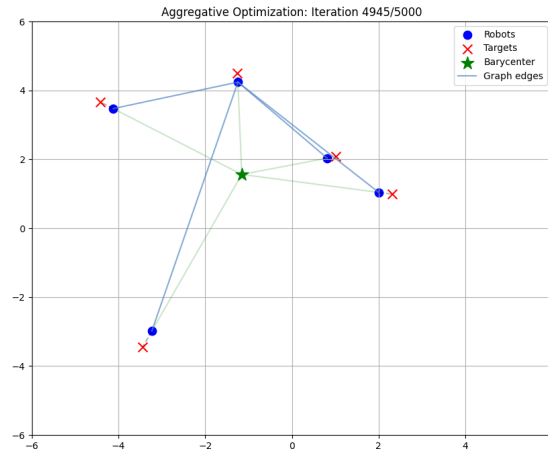
Consensus Error

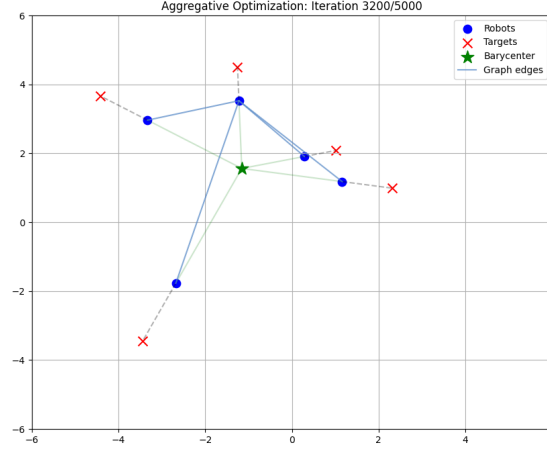
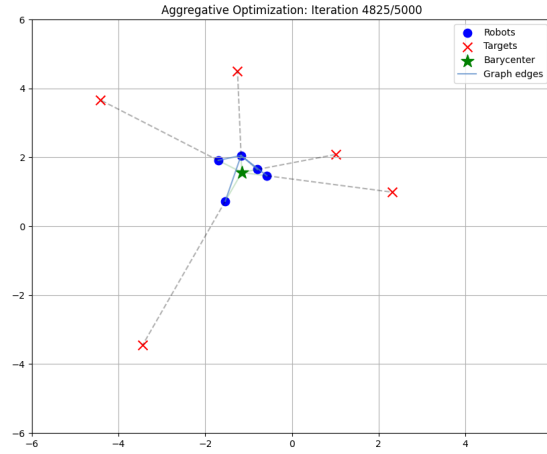
(a) $\beta = 0.1$ (b) $\beta = 5$

We can notice that increasing β makes the barycenter and the consensus errors converge faster. Increasing it even more leads to instability, and the metrics become oscillatory. A hint of that can be seen in 2.4b and 2.5b, where we can see some slight oscillations.

Final Positions Plot

These scatter plots are taken from the animation to show the motion of the robots and the barycenter.

Figure 2.6: Animation($\beta = 0.1$)

Figure 2.7: Animation($\beta = 0.5$)Figure 2.8: Animation($\beta = 5$)

Changing the parameter β , leads to the same barycenter, but different positions of the robots: when β is small the fleet is more dispersed and each robot is closer to its target (2.6), while when β is high the fleet is more cohesive (2.8). To have a perfect balance between the two goals, we can set $\beta = 1$.

2.1.6 Conclusion

The aggregative tracking algorithm does its job, finding the right compromise for the robots between closeness to their targets and to their common barycenter. Being probability based, the ER is the least reliable. Also in this case, being more connected leads to better result. Moreover, the choice of β is crucial to decide how the robots will act. However, it is also important to not choose the value too high, as it may lead to instability.

2.2 Task 2.2 - ROS 2 Implementation

2.2.1 Workflow

The implementation follows the same steps as Task 2.1, but with the required variations to make it work for ROS 2. The parameters are the same as Task 2.1, to be able to make a comparison between the two. The architecture of the package is described in detail in the following sections. In particular, we have a launch file that, once launched, manages the robot and visualization nodes. In the launch file we can also select the important parameters that we may want to change, such as number of robots, α , β and the topology of the graph.

2.2.2 Robot Nodes

N of this nodes are launched, so that we can simulate N robots. This node is where the same algorithm as Task 2.1 takes place. With the communication protocol it is then possible to send the outcome to RViz

2.2.3 Visualizer Node and RViz

The visualizer node creates RViz markers for the following items:

Robots	Blue spheres
Targets	Red cubes
Barycenter	Little green sphere
Robot-Target connections	Orange lines

These are then sent to RViz, to have a 3D visualization of the system.

2.2.4 Communication Protocol

Robots exchange three state vectors using peer-to-peer message passing with *Float64MultiArray* messages. The three vectors are: current position z_i^k , aggregative tracker s_i^k and gradient tracker v_i^k .

These are essential to run the aggregative tracking algorithm.

2.2.5 Simulations

In this section, some simulations are performed varying the graph topology.

Total Cost Evolution

This metric describes the evolution of the cost J over the iterations:

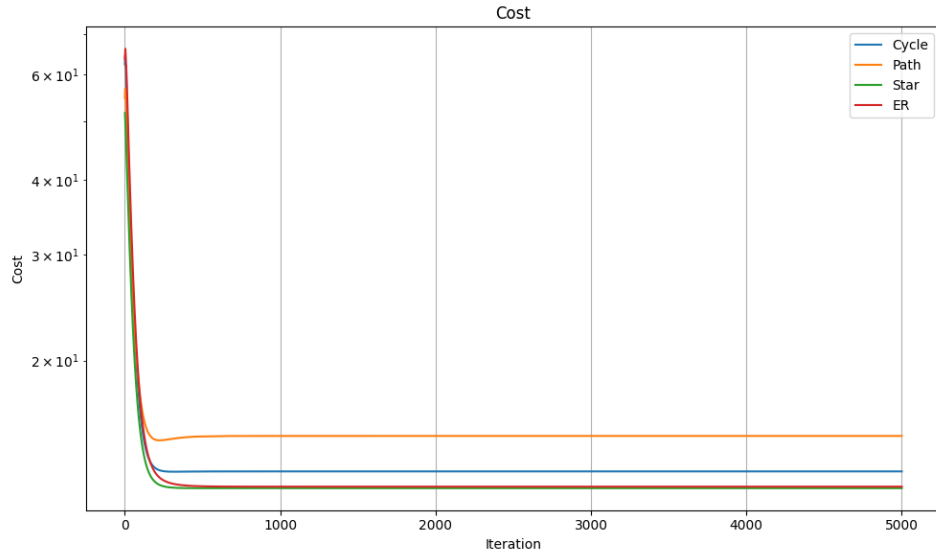


Figure 2.9: Total Cost Evolution

There are no significant differences in the convergence time. Yet, path topology converges to a higher cost with respect to the other topologies. Probably this is due to the fact that I run different simulations, launching the launch file four times (one for each graph type). In any case, the result looks consistent with the theory.

Global Gradient Norm

This metric describes the norm of the gradient $|\sum_i \nabla \ell_i(z_i)|$ over the iterations:

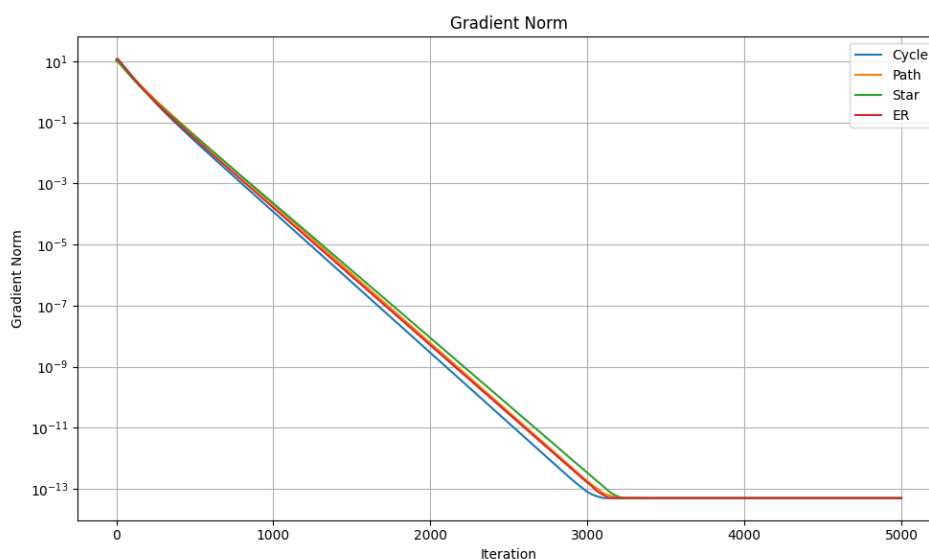


Figure 2.10: Total Gradient Norm

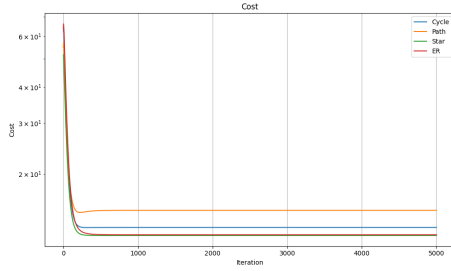
Also in this case, the result is correct, according to the theory.

2.2.6 Comparison with Task 2.1

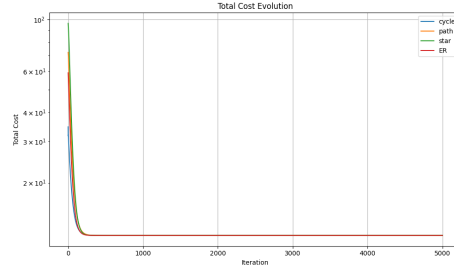
Now, we compare the results obtained in ROS2 with the ones from Task 2.1. We expect these values to be similar, if not equal.

Total Cost Evolution

This metric describes the evolution of the cost J over the iterations:



(a) ROS2 Implementation

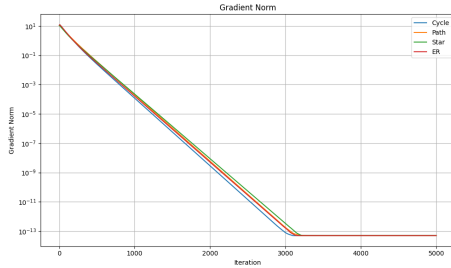


(b) Task 2.1 Implementation

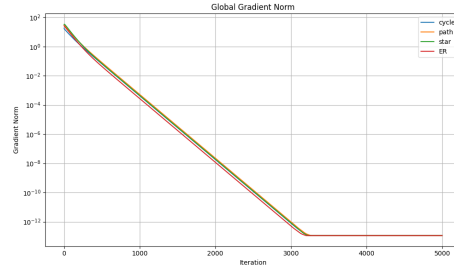
We can observe that the results are comparable, with only the initial cost being different. Still, the convergence happens in a similar number of iterations and at a similar value.

Global Gradient Norm

This metric describes the norm of the gradient $|\sum_i \nabla \ell_i(z_i)|$ over the iterations:



(a) ROS2 Implementation

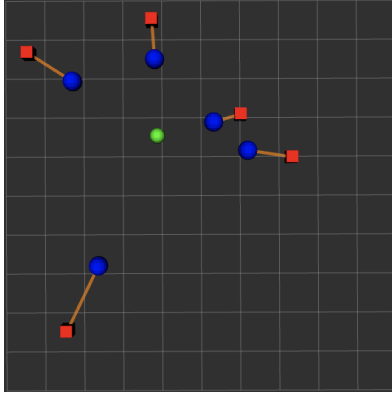


(b) Task 2.1 Implementation

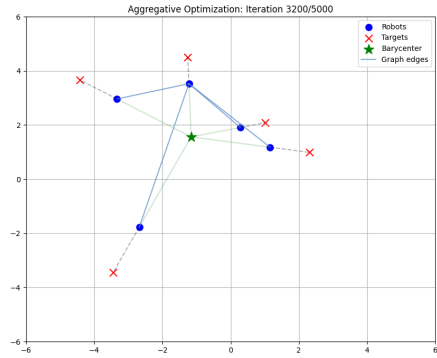
Again, the results are comparable. They also converge in a similar number of iterations and at a similar value.

Animation

In these two pictures we can observe that the final configuration of the fleet is the same between the two tasks. Both the animations were run with star topology.



(a) ROS2 Implementation



(b) Task 2.1 Implementation

In these two pictures we can observe that the final configuration of the fleet is the same between the two tasks. Both the animations were run with star topology.

2.2.7 Conclusion

The Task 2.1 has been successfully implemented in ROS2, as shown by the similar results.

Conclusions

The project has successfully shown the behavior of these algorithms in two different scenarios.

In the first task we were able to accurately localize, with a minimum error, N_T targets using N robots. We observed that the convergence is influenced by the graph connectivity and that the localization error lowers when the number of robots is increased.

In the second task we successfully balanced the robot between the private goal (staying close to a private target) and a collective one (staying close to the fleet barycenter). We discovered that tuning the tradeoff parameter β correctly is crucial to make the algorithm work regularly. Choosing a small β favors the private goal (so the fleet will be less compact and each robot closer to its private target), while a high β favors the collective goal. Although this is true, it may be counterproductive to use really high values of β , as this also introduces oscillatory motions of the robots.

The ROS 2 implementation allowed to validate the results obtained in the Python simulation.

Some future improvements to this project could include adaptive step-size techniques to possibly increase the convergence speed, using a large step-size in the beginning and then a smaller one as we get closer to the minima. This feature could be implemented using the Armijo rule (or simply a diminishing step-size). Another important extension could be the collision avoidance, both with other robots and with possible obstacles in the map. It would be necessary in a real application of this project, as some trajectories are not feasible right now. This feature could be implemented using safety filters, defining a minimum safety distance $\Delta > 0$ to keep between robots and obstacles.

Bibliography

- [1] G. Notarstefano and I Notarnicola. *Distributed Autonomous Systems Lecture Notes*. UniBO, 2025.